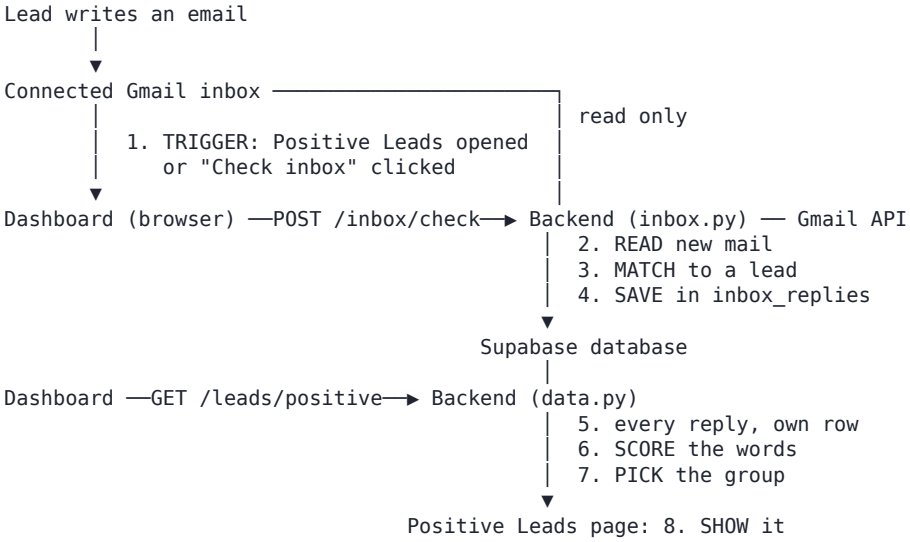
How Positive Leads works

From a lead's email to the Positive Leads page: how replies are received, scored and shown. Marketing Agent dashboard, October 2026.

The mail agent only **sends** outreach. Everything described here is done by this dashboard's own backend (`dashboardbackend/`).

The big picture



0. One-time setup (already done)

- **Gmail connected:** an admin clicked **Connect Gmail** on the History page. Google gave read-only access, and the key is saved encrypted in the `gmail_connection` table (migration 006).
- **Tables:** migration 007 created `inbox_replies` (the saved replies), `inbox_seen` (every message already looked at) and `reply_labels` (labels people choose).

1. Trigger: when mail is read

Nothing runs in the background. A check happens:

- when someone **opens Positive Leads**, at most once every 5 minutes per browser tab; or
- when someone clicks **Check inbox**.

The page calls `POST /inbox/check` on the backend.

2. Read new mail (inbox.py)

- The backend decrypts the saved key and gets a Gmail access token.
- It asks Gmail for messages in the **Inbox**, from the day before your first outreach email onwards (no reply can be older).
- It skips messages already in `inbox_seen`, unless the reply saved from one was deleted since; then it is read again.
- It reads up to **25 new messages** per check, oldest first. The rest wait for the next check.

3. Decide what each message is

Message	Result
Sent by the connected account itself	**Own mail**: ignored
From mailer-daemon / postmaster	**Bounce**: saved and marked bounced
Answers one of our emails	**Reply**: saved
Out-of-office (from its headers or subject)	Saved, but marked ooo and hidden
Matches nothing of ours	**Not a reply**: ignored

“Answers one of our emails” is checked three ways, strongest first:

- **Headers:** the email says “In-Reply-To: <our email's id>”.
- **Thread:** it is in the same Gmail conversation as an email we sent.
- **Sender:** it comes from the address of a lead we emailed, even as a brand-new email.

The test lead works through the third way: the email comes from `shivam714bhandari@gmail.com`, which is Test Lead's address, and Test Lead has a `sent` email.

4. Save it

- A reply goes into `inbox_replies`: who sent it, the text, when, which lead, which of our emails it answers, and how it was matched.
- Every message, reply or not, is recorded in `inbox_seen` so it is never read twice.
- The check also opens up to 10 threads to see whether **we answered** later (a newer message from us in the same conversation) and records `we_replied_at`.
- The page shows, for example: “Checked 14 emails in Gmail since Oct 6: 1 new reply from leads.”

5. Load the page (GET /leads/positive, data.py)

The backend takes **every reply written by a person** from leads (a contact joined to its company), skipping out-of-office and bounces, plus any label a person gave it. **Each reply is its own row**, judged on its own words: if a lead said “yes” on Monday and “no thanks” on Friday, both rows are listed, the Monday one in Positive and the Friday one in Not interested. A new email adds a row; older rows stay where they are.

6. Score the words (score_reply() in reply_rules.py)

1. **Clean the text:** our quoted email (“On Mon, … wrote:”, lines starting with “>”) is removed, so only what the lead wrote is scored.

2. **Start at 0.50.**

3. **Short replies:** if the whole email is a few words, a line that is just “Sure.” or “Interested!” counts as a clear yes (+0.25), and “No.” as a clear no (−0.30).

4. **“No” phrases first, then “yes” phrases.** Each phrase found adds its weight and is blanked out, so it cannot be counted twice:

Kind	Weight	Examples
Strong no	−0.30	not interested, no thanks, already work with, not a fit
Soft no	−0.20	not now, not a priority, unfortunately
Strong yes	+0.25	let's talk, send me the details, sounds great, what time works
Soft yes	+0.15	sounds interesting, tell me more, interested

Because the “no” phrases go first, “not interested” can never also count as “interested”.

5. **Cap the score** between 0 and 1, and note whether the email asks a question (a “?”, “how much”, “pricing”, …).

Example: “Sounds great, send me some times.” → 0.50 + 0.25 (“send me some times”) + 0.25 (“sounds great”) = **1.00**.

7. Pick the group (_judge() in data.py)

Checked in this order; the first rule that applies wins:

#	Rule	Group
1	A person chose a label in the drawer	That label, always
2	It asks to unsubscribe or stop (“remove me”, “stop emailing”)	**Unsubscribe**
3	It has both a yes and a no phrase	**Needs review**
4	Score 0.70 or more	**Positive**
5	It asks a question and has no “no” phrase	**Questions**
6	Score below 0.40	**Not interested**
7	Anything else (0.40 – 0.70, or no phrases at all)	**Needs review**

Follow-up status:

- **We replied:** we answered in the thread.
- **No answer needed:** a “no”.
- **Wrote again:** an older email; the lead wrote again since, and the newer one is the one waiting.
- **Waiting N days:** anything else; it turns red after 3 days.

Leads waiting for us are listed first, longest-waiting on top. Nothing is stored: the score is worked out fresh every time the page loads, so editing the phrase list re-scores every reply at once.

8. Show it on Positive Leads

- **Group tabs** with counts: Positive, Needs review, Questions, Not interested, Unsubscribe.
- **Each row** is one email: the lead, when they wrote, follow-up status, **Why Here** (either “Labelled by a person”, or the score plus the phrases that matched), and a short preview of the reply.
- **The drawer** (click a row): the full reply, the **Keyword Score** row with every matched phrase, and **Label This Reply**.
- **Top cards** count leads, not emails: Positive Leads (leads with a positive email), Leads Replied (every lead who wrote back) and Positive Rate (positive ÷ replied).

9. People correct it

If the keywords get one wrong, someone picks the right label in the drawer. It is saved in `reply_labels` for that one email, its row moves at once, and that label always wins over the score. **Remove** gives the decision back to the score. The mail agent never sees these labels, so an unsubscribe still has to be handled in the agent.

What never happens

- The dashboard never **sends**, deletes, archives or marks Gmail mail as read.
- It never writes to the agent's tables (`emails`, `contacts`, …).
- Mail from people we never emailed, or archived mail, is not picked up.

Where it lives in the code

What	File
Read Gmail, match, save	dashboardbackend/app/inbox.py
Gmail API calls, decrypt the key	dashboardbackend/app/gmail.py
Keyword score, bounce / out-of-office / unsubscribe rules	dashboardbackend/app/reply_rules.py
Every reply, groups (_judge), /leads/positive	dashboardbackend/app/data.py
Positive Leads page	dashboard/components/PositiveLeadsView.js
Drawer (Keyword Score, Label This Reply)	dashboard/components/drawerKinds.js
Test emails and expected results	docs/test-emails.md, docs/send-test-emails.md